

TerryFox LIMS — Rapport de la version 2

Ce qui a été demandé, ce qui a été décidé, et pourquoi.

Période : 13 – 25 août 2026 · 10 incréments · 25 commits · 10 migrations · 91 tests

État : **intégralement déployé en production**

Comment lire ce rapport

Chaque demande du consortium a sa section, avec le même découpage : **ce qui était demandé**, **ce qui a été décidé**, **pourquoi**, et **ce qui a été livré**. Les décisions qui ont demandé un arbitrage sont signalées comme telles.

Trois sections encadrent ces demandes : la méthode de travail, les arbitrages du 13 août, et ce qui a été découvert en cours de route sans avoir été demandé.

Tous les chiffres cités ont été mesurés sur la base réelle, pas estimés.

1. La méthode : rien avant les garde-fous

La contrainte « aucune donnée ne doit être perdue » a déterminé l'ordre de tout le chantier. Avant la première modification fonctionnelle, l'audit a montré que la base de production était dans une situation où une seule fausse manœuvre suffisait à perdre des mois de saisie :

- **aucune sauvegarde n'existait** — ni cron, ni timer, ni fichier ailleurs que sur le disque ; la seule copie hors machine était celle du dépôt git, dont les deux derniers commits étaient espacés de neuf mois ;
- `db.sqlite3` **était suivi par git et perpétuellement modifié**, gunicorn écrivant dedans en continu : un `git checkout .` dans le répertoire de production remplaçait la base vivante par celle du dernier commit, soit un recul de près d'un an ;
- **le watchdog redémarrait le service toutes les 5 minutes** dès que la sonde échouait, y compris au milieu d'un `migrate`, donnant à trois workers un accès concurrent à une base à moitié migrée.

Décision : construire le filet avant de sauter

L'incrément 1 n'a livré aucune fonctionnalité. Il a posé huit mécanismes en couches, dont chacun rattrape ce que le précédent laisse passer :

MÉCANISME	CE QU'IL EMPÊCHE
Base hors de l'arbre git (<code>/var/lib/terryfox-lims/</code>)	Aucune commande git ne peut plus atteindre les données vivantes
Sauvegarde horaire vérifiée, rétention 48 h / 30 j / 12 mois	Perte maximale d'une heure
Refus de migrer sans point de restauration frais	Une migration sans filet devient impossible

MÉCANISME	CE QU'IL EMPÊCHE
Contrôle d'invariants bloquant, avant et après	Un écart non déclaré annule et restaure automatiquement
Migrations réversibles, testées dans les deux sens	Le retour arrière est un chemin éprouvé, pas une intention
Suppression douce	Rien ne quitte la base par une interface web
Cellule vide = inchangé, jamais effacé	Un import partiel ne détruit plus de données
Restauration réellement répétée	Une sauvegarde jamais restaurée n'est pas une sauvegarde

Chaque sauvegarde est relue et recomptée juste après création ; si les comptages ne correspondent pas à la source, le fichier est supprimé et le processus sort en erreur. Une sauvegarde non vérifiée inspire une confiance qu'elle ne mérite pas.

L'exercice de restauration a eu lieu pour de vrai

Le premier déploiement de la migration 0019 a déclenché une **restauration automatique complète sur la base de production**, à cause d'un faux positif du contrôle d'invariants. Elle a fonctionné : base remise en état, service redémarré, invariants conformes. La procédure de retour arrière est donc vérifiée en conditions réelles, pas seulement écrite.

Une suite de tests, là où il n'y en avait aucune

Le dépôt ne pouvait pas en avoir : la migration `0002_swap_pi_bioinfo_permissions` appelait `Group.objects.get(name='PI')` sans garde-fou, alors qu'une autre migration renomme ce groupe. **Aucune base ne pouvait être créée de zéro**, donc aucune base de test. Corrigé — la migration étant déjà appliquée en production, le correctif ne concerne que les bases neuves. C'est aussi ce qui a rendu possible l'archive V1.

La suite compte aujourd'hui **91 tests**, plus deux outils qui se contrôlent eux-mêmes : `ops/selftest.py` (le contrôle d'invariants) et `ops/lint_templates.py` (les motifs qui cassent sur téléphone).

2. Les arbitrages du 13 août

Quatre points ont été tranchés en cours de route et ont réorienté le plan.

2.1 Le dépôt reste public, pas de réinitialisation des mots de passe

Décision retenue, au motif qu'il n'y a pas eu d'incident de sécurité.

Cette position tient techniquement : les hachages sont du PBKDF2-SHA256 à 600 000 – 870 000 itérations, et les mots de passe distribués par le LIMS font 12 caractères tirés sur 62. Les casser est hors de portée pratique.

Trois éléments distincts subsistaient, signalés une fois :

1. La `SECRET_KEY` publiée permet de **forger une session Django valide** sans connaître aucun mot de passe. C'est indépendant de la robustesse des mots de passe.
2. `_generate_password()` utilise `random` et non `secrets` — un générateur non cryptographique.
3. `db.sqlite3` publiait 1329 cas de recherche avec leurs Biobank ID. Question de gouvernance de données du consortium, pas de sécurité.

Ce qui a été fait : l'action n° 1 des garde-fous — sortir la base de l'arbre git — retire mécaniquement les données du dépôt. Le code reste public, la base cesse d'être publiée. Les deux exigences ne s'opposaient pas.

2.2 La V1 doit rester accessible

Décision de conception : la figer, pas la brancher sur les données vivantes.

La tentation aurait été de garder les anciennes vues actives sur la base à jour, sous un préfixe `/v1/`. Cela ne tient pas : le code V1 lit `other_id`, ignore les spécimens, les tentatives archivées et la suppression douce. Sur des données courantes il afficherait des statuts vides et des cas incomplets — **une V1 qui ment est pire que pas de V1**. Et chaque migration future obligerait à maintenir deux jeux de vues sur un même schéma.

Ce qui a été livré : le code du commit `v1.0-final` sur un instantané des données d'avant migration, servi sur le port 8443 avec son propre certificat, en lecture seule sur trois couches :

1. fichier en `444`, propriétaire root ;
2. SQLite ouvert en `mode=ro` — toute écriture refusée par le moteur ;
3. middleware rejetant tout ce qui n'est pas GET ou HEAD, avec une page d'explication plutôt qu'une erreur 500.

Deux pièges silencieux ont été trouvés à l'essai, tous deux rendant l'archive inutilisable : les sessions Django s'écrivent en base (plus personne ne pouvait se connecter), et Django met à jour `last_login` à chaque connexion. Le débranchement de ce dernier exige de repasser `dispatch_uid='update_last_login'` — sans lui, `disconnect()` ne trouve rien et ne fait rien, sans le dire. L'installateur contrôle désormais qu'on peut encore se connecter.

2.3 Aucune donnée perdue

Traité en section 1. **Résultat mesuré :** à travers dix migrations dont deux remplissages massifs, la base est passée de 1329 cas à 1329 cas, et la distribution des tiers de 552 A / 682 B / 95 FAIL à 552 / 682 / 95 — au chiffre près.

2.4 L'interface doit être élégante

Traité en section 3.10. L'archive V1 servant de filet, la refonte a pu être menée franchement plutôt que prudemment.

3. Les demandes du consortium

3.1 · ACC généré par le LIMS, Biobank ID cherchable, type de spécimen

Demandé — L'ACC est généré automatiquement par le LIMS. Le Biobank ID sert à rechercher. L'utilisateur choisit parmi trois types de spécimen.

Décidé

Le compteur d'ACC est **monotone et ne réutilise jamais un numéro libéré**. Les 169 trous entre 1 et 1498 restent des trous : un ACC retiré peut déjà figurer sur une étiquette de congélateur ou un dossier papier, et le réattribuer ferait silencieusement porter le même identifiant à deux patients différents.

L'allocation fait **UPDATE puis SELECT**, dans cet ordre. La première instruction prend le verrou d'écriture de SQLite et le conserve jusqu'au commit, donc deux allocations concurrentes se sérialisent au lieu de s'entrelacer. `Max(acc_number) + 1` serait en situation de course avec trois workers gunicorn, et `select_for_update()` lève `NotSupportedError` sur SQLite. Vérifié : 60 allocations sur trois fils, aucune collision.

`other_id` devient `biobank_id`, indexé. **La recherche a dû suivre** : l'unique champ de la page projet interrogeait `Case.name`, devenu une chaîne générée — taper « N-BBN 440 » n'aurait rien renvoyé, et la demande phare aurait été ratée par omission. Le champ interrogé désormais les deux identifiants, le paramètre `?name=` est conservé pour ne pas casser les signets, et une recherche transverse tous projets arrive dans la barre de navigation.

Insight structurant — Les « trois types de spécimen » de cette demande et la découpe Normal / Tumeur-ADN / Tumeur-ARN demandée par Daniel **sont le même concept**. Un seul objet nouveau à introduire dans l'interface, pas deux.

Livré — `IdentifierSequence`, `Case.acc_number`, `Case.biobank_id`, recherche unifiée, recherche globale / `search/`, choix des types de spécimen à la création.

Risque écarté — `makemigrations` avait généré `RemoveField('other_id') + AddField('biobank_id')`, ce qui aurait **effacé les 1329 Biobank ID**. La migration 0020 a été écrite à la main avec `RenameField`. Vérification faite non sur un échantillon mais sur l'ensemble complet : **0 valeur modifiée, multiensemble identique, 0 nom de cas changé**.

3.2 · Cas prioritaires

Demandé — Un indicateur oui/non posé à la création, à la discrétion de la biobanque, pour les patients dont le pronostic impose d'agir vite.

Décidé — Les cas marqués **remontent en tête de chaque liste**. Un cas urgent sorti de l'écran par le défilement vide la demande de sa substance. Le test le vérifie sur un cas dont l'ACC est plus grand, pour que seul l'épinglage puisse expliquer l'ordre obtenu.

Traitement visuel : un filet ambre et un fanion. **Jamais une ligne rouge** — le rouge signale l'échec, pas l'urgence, et un mur de rouge ne se lit plus.

Livré — `Case.is_priority`, épinglage, filtre « Priority only », pose à l'unité ou sur tout un lot.

3.3 · Changement de statut en lot

Demandé — Des cases à cocher à côté de chaque cas, pour passer plusieurs dizaines de cas d'un statut à l'autre en quelques clics, au lieu de télécharger un CSV puis de le re-téléverser.

Décidé

La grille de cartes devient un **tableau dense**. Cocher quarante lignes parmi cent vingt-huit rangées de cartes, sans tri ni sélection par plage, aurait remplacé l'aller-retour CSV par pire.

Deux menus, pas un : « passer 40 cas à Sequencing Complete » est ambigu dès lors que le statut vit sur le spécimen — il faut dire à quoi on l'applique.

L'annulation ne rend un spécimen que **s'il est encore à la valeur posée par le lot**. Un spécimen modifié depuis est laissé tel quel : annuler ne doit pas écraser le travail fait après. Vérifié en conditions réelles : sur 6 spécimens, 5 restaurés et 1 laissé intact, avec le message qui l'explique.

Livré — Tableau, cases à cocher, sélection par plage au Maj-clic, barre d'action collante, journal ligne par ligne (`BatchOperation` + `SpecimenStatusChange`), annulation.

Environ 60 lignes de JavaScript nu : aucune dépendance, aucune étape de compilation — le projet n'a ni npm ni bundler. **Et sans JavaScript la fonctionnalité marche quand même**, des cases à cocher et un bouton d'envoi étant du HTML natif.

3.4 • Re-soumission

Demandé — Quand un séquençage échoue, la biobanque soumet un nouveau spécimen pour le même patient. Le système génère un nouveau cas avec le **même ACC**. L'historique de l'ancien est archivé.

Décidé — résolution d'une contradiction

Cette demande contredit littéralement la prévention des doublons (§ 3.6) : l'une veut le même identifiant, l'autre l'interdit.

L'unicité porte sur la tentative en cours. Un index unique partiel conditionné sur `is_archived = False` autorise exactement un cas actif par ACC, les tentatives archivées le partageant librement. Comportement vérifié sur SQLite 3.45.

On archive avant de créer. Créer d'abord ferait exister deux cas actifs avec le même numéro, ne serait-ce qu'un instant, et la contrainte tomberait.

Rien n'est copié. Commentaires, couvertures et statuts restent physiquement attachés à la tentative archivée — aucune copie, donc aucune perte possible dans la copie. C'est littéralement ce que demande « l'historique de l'ancien cas est archivé ».

Un **report sélectif** permet de garder les spécimens encore bons : quand seul l'ARN a échoué, le Normal conserve sa couverture et son statut.

Une distinction a été posée dans l'interface : re-soumettre suppose un **nouveau spécimen physique** ; relancer le séquençage du même spécimen est un top-up, qui se fait en reculant le statut sur le même cas. Sans cela les deux gestes se confondent et l'historique se remplit de fausses tentatives.

Livré — `Case.resubmit()`, page de confirmation, bandeau de lecture seule sur les tentatives archivées (toujours consultables par leur URL — un vieux lien doit continuer de mener quelque part), panneau des tentatives précédentes avec leur nombre de commentaires, là où se trouve généralement la raison de la re-soumission.

3.5 • Exports

Demandé — Export complet des données et métadonnées pour un projet, et export global de tous les projets pour le consortium.

Décidé — la forme du fichier est choisie pour ce qu'on en fait

`cases.csv` est **large** : une ligne par cas, un bloc de colonnes fixe par spécimen. C'est ce format qu'un PI croise avec sa feuille clinique par RECHERCHEV sur le Biobank ID. Un fichier long, une ligne par spécimen, **triplerait son effectif sans qu'il s'en aperçoive**. La granularité fine est servie par `specimens.csv`, dans un fichier séparé.

Deux pièges de correctitude désamorcés :

1. **Les tentatives archivées ont leur propre fichier**, jamais mélangées aux cas actifs derrière une colonne d'état — une erreur de filtre gonflerait un effectif publié.
2. **Chaque fichier enfant porte** (`ACC`, `Attempt`), et le README l'explique en toutes lettres : joindre sur le seul ACC après une re-soumission dupliquerait les lignes de la tentative 1 sur la tentative 2.

Un spécimen absent se lit `not collected`, pas case vide — pour distinguer « pas prévu au protocole » de « en attente ».

Livré — Export CSV par projet, lot ZIP (`cases`, `specimens`, `comments`, `cases_archived`, `README.txt`), export consortium. Bibliothèque standard uniquement, aucune dépendance ajoutée.

Permission — L'export consortium réunit les données de tous les groupes : superutilisateurs seulement, journalisé en WARNING avec l'auteur et son adresse. Les PI gardent l'export de leur projet.

Vérifié : un commentaire contenant sauts de ligne, virgules et guillemets est relu en une seule ligne CSV — 1553 pour 1553. Le nombre de requêtes ne dépend pas du nombre de cas.

3.6 · Prévention des doublons

Demandé — Le système bloque les doublons ; un même identifiant ne peut plus être créé dans deux projets différents par erreur.

Décidé — deux régimes différents, délibérément

IDENTIFIANT	RÉGIME	MOTIF
ACC	Unicité dure , contrainte de base de données	Il est généré par le LIMS : la contrainte est gratuite, et 0 doublon existait déjà
Biobank ID	Contrôle souple , dans le formulaire	Voir ci-dessous

Le contrôle souple sur le Biobank ID a été choisi après examen de la donnée : **P08_CRC utilise des identifiants numériques nus de 5 à 849, et P09_BC_EV de 102 à 850**. Deux projets partagent le même espace de numérotation. Zéro collision aujourd'hui relève de la chance, pas de la conception. Le jour où P09 enregistre son patient 300 alors que P08 a déjà un 300, une contrainte dure bloquerait la technicienne sur le **vrai** identifiant du patient.

La demande dit « ne peut plus être créé dans deux projets **par erreur** ». C'est une prévention d'erreur, pas un axiome d'unicité.

Livré — Contrainte DB sur l'ACC. Sur le Biobank ID, un message qui **nomme le cas en conflit et son projet** — `Biobank ID "N-BBN 42" is already used by ACC-0311 in P06...` — avec un bouton pour passer outre. Jamais un nom de contrainte SQL.

3.7 · Projet « Referred Cases »

Demandé — Une catégorie distincte pour les cas urgents référés par des médecins, hors des projets de recherche de la phase 1.

Décidé — `Project.kind` sépare les projets de recherche de cette catégorie. Le projet est créé par migration ; le retour arrière ne le supprime que **s'il est resté vide** — on ne détruit pas des cas en revenant sur une migration.

Livré — Le projet existe en production, avec son type distinct visible dans la liste.

3.8 · Nouveaux statuts, en trois étapes

Demandé — Biobanque : Case Created, Sent to Sequencing Center. Séquençage : Received, QC complete, Library complete, iSeq finished, Sequencing Complete. Bio-informatique : Transferred to CAIR, Analyzing, Analysis complete.

Décidé

Le regroupement en trois étapes **porte l'ordre**, il n'est pas décoratif. Dix badges de couleurs différentes n'apprennent à personne que « QC complete » précède « Library complete » ; une liste déroulante groupée, si.

Le problème des 469 cas sans équivalent. `incomplete` (428) et `unknown` (41) — 35 % de la base — n'existent dans aucune des trois étapes. La donnée a tranché à moitié la question : **385 des 428 cas `incomplete` ont leurs deux couvertures ADN et pas d'ARN**, et leurs commentaires disent « Needs Normal Top-Up », « No DNA Normal fastq ». `incomplete` signifie donc « ARN en attente » — ce que le modèle par spécimen exprime nativement.

Décision : les conserver comme statut de transition `Unknown (pre-V2)`, invisible à la saisie mais **présent dans les filtres**. Sans cela, le personnel ne pourrait plus retrouver les cas qu'on lui demande de reclasser. C'est la file de travail que l'outil de lot est fait pour vider — **439 cas** aujourd'hui.

Livré — `core/statuses.py` : 10 statuts, 3 étapes, rangs espacés de dix pour pouvoir insérer une étape sans tout renuméroter. `statuses.from_any()` accepte les slugs et les libellés v1, pour que les fichiers CSV existants restent importables.

3.9 · Spécimens séparés dans un cas (Daniel, TFRI)

« Things should be organized by case but tumours and normals should be separate entities inside a case. And the tumour needs to be split into the DNA and the RNA. Faut être capable de suivre ces choses-là indépendamment même s'ils ont le même case ID. »

C'est la demande la plus structurante du chantier, et la seule migration difficilement réversible.

Décidé

Les trois entités correspondent **une pour une** aux trois colonnes de couverture existantes : `dna_n` → Normal-ADN, `dna_t` → Tumeur-ADN, `rna` → Tumeur-ARN. La migration est donc sans perte, et les seuils de tier se transposent sans changer une valeur.

Les colonnes d'origine sont conservées comme miroir, rafraîchies à chaque écriture. C'est ce qui laisse `calculate_tier()` intact — et rend la migration prouvablement neutre sur les tiers.

Le statut descend au spécimen, mais l'interface continue de parler cas. S'y arrêter aurait fait passer l'action la plus fréquente du système de un menu déroulant à trois. Le chemin par défaut est donc un seul menu qui applique à tous les spécimens ; le réglage fin par spécimen est replié en dessous.

Une erreur rattrapée en cours de route. Ma première version du rollup ignorait purement les spécimens à reclasser, ce qui faisait passer **1315 cas en « Analysis complete »** contre 855 auparavant : les 428 cas `incomplete` se déclaraient terminés. Un PI aurait lu ça comme « tout est fait ». Le statut du cas vaut désormais celui du spécimen le moins avancé **parmi ceux dont l'état est connu** — les 855 cas réellement terminés ne régressent pas — et les spécimens en attente sont comptés à part, affichés en pastille à côté du statut, avec un filtre dédié.

Aucun cas n'est forcé à trois spécimens. Le formulaire fait choisir les types, et la migration déduit **de la donnée** les projets sans ARN plutôt que de les coder en dur.

Arbitrage demandé et rendu — Sur la base actuelle, un seul projet n'a aucune valeur d'ARN : **P10_Prostate, 32 cas sur 32**. Décision du consortium : son protocole ne prévoit pas d'ARN, il reçoit donc deux spécimens par cas. Lui en fabriquer un troisième l'aurait laissé « en attente de Tumeur (ARN) » à perpétuité.

Une hypothèse de l'analyse initiale a été corrigée par la donnée : P01_Lung était décrit comme du ctDNA sans ARN ; il a en réalité de l'ARN sur 79 % de ses cas (120/151).

Livré — 3955 spécimens (1329 × 3 – 32), suivi indépendant, un menu unique pour faire avancer le cas, panneau par spécimen replié, barres de progression à trois segments.

CONTRÔLE APRÈS MIGRATION	RÉSULTAT
Couverture de spécimen vs colonne du cas	0 écart sur 3955
Distribution des tiers	552 / 682 / 95 — identique
Cas à 3 spécimens / à 2	1297 / 32

3.10 · Refonte visuelle

Demandé — Une interface plus belle et plus moderne, sans « slops AI », et qui reste facile à utiliser.

Décidé

L'interface n'était pas laide, elle était datée d'une façon précise : `base.html` contenait 265 lignes de CSS en ligne reprenant la palette Flat UI Colors de 2013, avec une ombre et un survol « lévitant » sur **chaque** carte, y compris celles qui ne sont pas cliquables — le signal « kit de template » le plus fort de l'application, et un mensonge sur ce qui est cliquable.

Principes retenus :

- **Bordures, jamais d'ombre au repos.** L'ombre est réservée aux couches réellement flottantes. Un système à filets s'imprime, une ombre non.
- **Une teinte par étape de statut, pas par statut.** Quatre couleurs à apprendre au lieu de dix, et immunisé contre le prochain changement de vocabulaire.
- **Identité typographique.** IBM Plex Sans et Plex Mono, auto-hébergées. Les identifiants passent en chasse fixe : `ACC-0142` et `ACC-0412` se distinguent en mono, pas en proportionnel.
- **Rien ne change de place.** Barre de navigation en haut, fil d'Ariane, action principale en haut à droite, toutes les URL et tous les paramètres de filtre préservés. Une barre latérale serait plus photogénique et coûterait un réapprentissage à 42 personnes pour cinq destinations.
- **Pas de mode sombre.** Personne ne l'a demandé, il doublerait la surface de vérification visuelle, et un mode sombre non sollicité fait partie des signes du travail généré. C'est une décision, pas un oubli.

Accessibilité — un cas où la mesure a contredit l'intention. Les anciens badges tenaient entre **2,10 et 3,82 : 1** de contraste, tous sous le seuil WCAG AA de 4,5. Les nouveaux sont à 5,20 – 5,93 : 1. Et la sémantique du tier était **inversée** : A en rouge, FAIL en gris — un PI qui survolait un projet voyait un mur de rouge et lisait une catastrophe.

En validant la palette, le constat suivant s'est imposé : **aucun triplet vert/ambre/rouge ne passe le contrôle de daltonisme**. Quatre variantes ont été essayées (cramoisi, ambre plus jaune, sarcelle, magenta) ; l'ambre et le rouge restent indistinguables en deutéranopie — c'est le problème classique du feu tricolore, pas un défaut de nuance. La réponse n'est donc pas une couleur magique mais **l'encodage secondaire** : le tier porte toujours sa lettre (A / B / FAIL), en pastille carrée distincte des statuts, et les barres du tableau de bord n'encodent la magnitude que par une seule teinte.

Livré — `static/css/lims.css` piloté par jetons, grille de 4 px, accueil transformé en tableau de bord, états vides dessinés, feuille d'impression, focus visible partout, `prefers-reduced-motion` respecté. Bootstrap, FontAwesome et les polices **vendorisés** : plus aucune requête vers un CDN — un serveur de laboratoire interne ne doit pas dépendre d'internet pour s'afficher.

4. Ce qui a été trouvé sans avoir été demandé

Ces points n'étaient dans aucune demande. Ils ont été découverts en auditant le code et le serveur, et corrigés.

Pertes de données actives

TROUVAILLE	CONSÉQUENCE
L'import CSV effaçait les couvertures sur cellule vide	Et comme <code>Case.save()</code> recalcule le tier, les cas basculaient silencieusement en FAIL . Démonstration sur ACC-0042 : de tier A à FAIL
Supprimer un projet détruisait ses cas	Mesuré sur P06 : le projet plus ses 256 cas et leurs commentaires, derrière une simple page de confirmation, sans corbeille ni sauvegarde
Aucune sauvegarde n'existait	Voir § 1

Configuration inerte ou fragile

TROUVAILLE	CONSÉQUENCE
<code>STATICFILES_STORAGE</code> retiré en Django 5.1	Le réglage whitenoise était toujours présent, silencieusement sans effet depuis la montée de version : fichiers statiques servis sans compression, sans hachage, sans cache. Migré vers <code>STORAGES</code> — la feuille passe désormais de 28 Ko à 6,6 Ko en Brotli , avec un cache immuable
<code>pip install gunicorn</code> à chaque démarrage	9,87 s de réseau dans le chemin critique, alors que le watchdog relance le service toutes les 5 minutes en cas de panne : une coupure réseau devenait une boucle de redémarrages lents. Rendu conditionnel — 0,02 s
Deux modules de réglages intégralement dupliqués	150 réglages dont 130 identiques ; toute modification devait être faite deux fois. Mis en couches, vérifié par comparaison des 150 réglages résolus : 0 écart en production

Performance

La page du plus gros projet (P06, 256 cas) déclenchait **522 requêtes SQL** — deux par cas, dues à `case.accessions.count` et `case.comments.count` dans la boucle du gabarit.

	AVANT	APRÈS
Requêtes SQL	522	11
Temps	738 ms	227 ms
Poids HTML	926 Ko	374 Ko

Le test ne vérifie pas un nombre absolu mais l'invariant : un projet de 3 cas doit générer autant de requêtes qu'un projet de 250.

Mobile

Signalé en fin de chantier : l'interface débordait sur téléphone « à beaucoup d'endroits ».

La cause était unique et structurelle : la feuille de style n'avait **aucun point de rupture de largeur**. Chaque rangée flex était une chaîne d'atomes irréductibles — les enfants flex ont `min-width: auto` — et poussait la page au lieu de se replier. L'en-tête de la page projet réclamait à lui seul **~1050 px dans un viewport de 360**.

Un audit de tous les gabarits a confirmé **38 problèmes**, dont 4 faisant défiler la page. 43 correctifs appliqués. **Seules quatre déclarations touchent le grand écran**, et chacune corrige un défaut réel — dont `.alert-dismissible`, dont la règle `.alert` écrasait le `padding-right` de Bootstrap : le texte des messages passait sous la croix de fermeture, sur chaque page.

Deux découvertes au passage : les champs sous **16 px déclenchent le zoom automatique de Safari iOS** à la mise au point, et un commentaire `{# ... #}` sur **plusieurs lignes** n'en est pas un pour Django — il partait tel quel dans le HTML.

Divers

- L'archive V1 était tuée à chaque démarrage du service principal, leurs lignes de commande se ressemblant (`wsgi_archive` contre `wsgi_prod`). Les motifs `pgrep/pkill` visent désormais `wsgi_prod` explicitement.
- Le certificat auto-signé du serveur avait **expiré le 2 juillet 2026**.
- Le code mort a été retiré : une API DRF jamais branchée au routeur (et `rest_framework` n'est même pas installé), et des gabarits React pointant vers un répertoire `frontend/` inexistant.

5. Chiffres

La base, avant et après

MESURE	AVANT	APRÈS
Projets	15	16 (+ Referred Cases)
Cas	1 329	1 329
Spécimens	—	3 955
Commentaires	1 550	1 550
Utilisateurs	42	42
Tiers A / B / FAIL	552 / 682 / 95	552 / 682 / 95
Orphelins, doublons d'ACC	0	0

Le chantier

Incréments livrés	10 / 10
Commits	25
Migrations	10
Tests Django	91
Outils d'exploitation	11 scripts, ~1 660 lignes
Gabarits	23

Vérifications systématiques

Chaque déploiement passe par `ops/deploy.sh`, qui neutralise le watchdog, prend une sauvegarde étiquetée vérifiée, fige les invariants, migre, recompare, et **restaure automatiquement** en cas d'écart non déclaré. Aucune migration n'a été lancée à la main.

Trois contrôles gardent l'outillage lui-même :

- `ops/selftest.py` — six scénarios sur bases synthétiques : le contrôle d'invariants doit attraper une perte de lignes, une suppression de masse et une base corrompue, et **ne pas** bloquer une migration purement additive ;
- `StaticAssetTests` — rend chaque page avec le stockage de production, où un chemin `{% static %}` manquant lève ;
- `ops/lint_templates.py` — estime la largeur réclamée par chaque rangée non repliable.

Chacun a été vérifié en **réintroduisant volontairement le défaut** qu'il doit attraper.

6. Ce qui reste

À faire par le consortium

439 cas portent encore un spécimen hérité de la V1 dont l'état reste à établir. Le chemin est prévu pour : filtrer sur « To classify » dans une page projet, tout sélectionner, appliquer le vrai statut. L'annulation est disponible si un lot part de travers.

Décision en suspens

Faire tourner ou non la `SECRET_KEY`. Ce n'est pas une réinitialisation de mot de passe : chacun se reconnecte une fois avec son mot de passe actuel. Sans rotation, la clé publiée continue de permettre la forge de sessions valides. Coût d'une reconnexion pour 42 personnes, contre cette exposition.

Limite connue

L'archive V1 est accessible sur `https://10.220.115.67:8443/`. Pour une adresse publique en `/v1/`, il faudra une route sur le proxy CAIR, qui n'est pas administré depuis cette machine.

Points laissés ouverts par conception

- **Le mode sombre** n'a pas été fait — décision assumée, réversible.
- **PostgreSQL** n'est pas justifié à cette échelle : trois écrivains et trois lecteurs concurrents produisent 0 erreur de verrou, la base fait 1,6 Mo et se sauvegarde en quelques dizaines de millisecondes. Le mode WAL est activé.
- **Les permissions des groupes** `viewer` / `editor` restent posées en base via l'admin, et non dans le code. C'était déjà le cas ; le changer n'était pas demandé.

Rapport établi le 25 août 2026. Les chiffres proviennent de mesures sur la base de production et des journaux de déploiement, pas d'estimations.